

ResumeX - AI Resume Screening & Intelligent Candidate F

Comprehensive System Architecture & Core DSA Algorithm Documentation

Author: Pasula Shloka | Course: Data Structures & Algorithms Capstone | October 2026

1. Executive Summary

ResumeX is an AI-powered resume screening and intelligent candidate ranking platform inspired by modern talent acquisition platforms such as LinkedIn Recruiter, Workday HCM, and Ashby ATS. The system is designed to automate resume evaluation against targeted job descriptions using verified, explainable Data Structures and Algorithms (DSA) rather than opaque black-box AI models.

- Problem Statement: Recruiters receive hundreds of resumes per job opening. Manual screening takes 6-8 seconds per candidate, resulting in fatigue, bias, and overlooked qualifications.
- Deterministic DSA Foundation: ResumeX implements exact string matching, dynamic programming edit distance, global sequence alignment, suffix arrays with LCP, SHA-256 hashing, and greedy approximation optimization to deliver mathematically sound, transparent rankings.

2. Decoupled System Architecture

The architecture strictly separates the high-performance algorithmic backend from the modern recruiter user interface, maintaining modularity and independent scalability.

- Eclipse Java Backend (Port 8080): Pure Java SE 17+ HTTP Server (com.sun.net.httpserver) executing all 6 DSA algorithms, in-memory repository storage, and Apache PDFBox 3.0.8 PDF text extraction.
- VS Code Modern Frontend (Port 5173): React 19 + Vite Single Page Application designed with recruiter-centric workflows (Dashboard, Job Requisitions, PDF Screening, Candidate Directory with CSV export, Hiring Pipeline Kanban, Multi-Resume Comparator, and Duplicate Auditor).

3. Core Data Structures & Algorithms Suite

Every score and ranking produced by ResumeX is backed by actual algorithmic computation in the Eclipse Java project:

Algorithm	DSA Category	Primary Function	Time Complexity
KMP Matcher	String Matching	Exact skill & keyword extraction	$O(N + M)$
Levenshtein DP	Dynamic Programming	Typo tolerance & abbreviation matching	$O(N * M)$
Needleman-Wunsch	Sequence Alignment DP	Qualification order & career progression	$O(N * M)$
Suffix Array	String Indexing	Full-text substring search & indexing	$O(N \log^2 N)$
Kasai LCP Array	Array Processing	Longest common prefix for domain phrases	$O(N)$
SHA-256 Hashing	Cryptographic Hashing	Exact duplicate resume detection	$O(N)$
Greedy Approximation	Optimization Algorithm	Multi-criteria candidate score weighting	$O(K \log K)$

3.1 Algorithm 1: Knuth-Morris-Pratt (KMP) String Matching

Implemented in KMPMatcher.java. Uses a precomputed prefix function (pi table) to identify exact required skills (e.g., 'Spring Boot', 'Data Structures', 'SQL') across candidate text in $O(N + M)$ linear time without character backtracking.

3.2 Algorithm 2: Levenshtein Edit Distance (Dynamic Programming)

Implemented in EditDistance.java. Uses a 2D dynamic programming recurrence to quantify string distance between candidate skills and job requirements. Normalizes distance into a similarity percentage to seamlessly recognize spelling variants and typos (e.g., 'Jvaa' -> 'Java', 'ReactJS' -> 'React').

3.3 Algorithm 3: Needleman-Wunsch Global Sequence Alignment

Implemented in SequenceAlignment.java. Evaluates whether the candidate's career progression and listed technologies mirror the prioritization of the job description. Uses match bonus (+2), mismatch penalty (-1), and gap penalty (-1) to trace the optimal global alignment.

3.4 Algorithms 4 & 5: Suffix Array and Kasai's LCP Algorithm

Implemented in SuffixArray.java and LCPArray.java. Indexes all suffixes of the combined resume and job corpus. Kasai's algorithm derives the Longest Common Prefix array in linear O(N) time, identifying recurring domain collocations and multi-word project phrases.

3.5 Algorithm 6: Cryptographic SHA-256 Hashing & Deduplication

Implemented in Hashing.java and DuplicateDetector.java. Normalizes resume text to compute SHA-256 message digests for instant O(1) exact duplicate identification, alongside Jaccard token set similarity for near-duplicate revision tracking.

3.6 Algorithm 7: Greedy Approximation Optimization

Implemented in ApproximateMatcher.java. Solves multi-criteria candidate optimization by approximating the maximum qualification coverage density across competing skill requirements.

4. Weighted Composite Scoring Model

Final candidate match scores are calculated using a weighted linear combination of the 6 algorithmic evaluations:

Evaluation Component	Algorithm Used	Weight	Focus Area
Direct Skill Match	KMP String Matching	30%	Mandatory core requirements
String Match Density	Keyword Matching	20%	General technical vocabulary
Edit Distance	Levenshtein DP	15%	Typo & variant tolerance
Sequence Alignment	Needleman-Wunsch DP	15%	Experience & workflow sequence
Suffix / LCP Score	Kasai Algorithm	10%	Deep domain phrase overlap
Approximation Factor	Greedy Set-Cover	10%	Multi-objective density factor

Recruiter Recommendation Thresholds

Strong Match (>= 70%): Highly recommended for immediate technical review. Moderate Match (55% - 69%): Meets core criteria with minor preferred skill gaps. Low Match (< 55%): Substantial requirement gaps; candidate not recommended.

5. Key Recruiter Modules & Workflow Features

- Job Openings Hub: Manage open requisitions with required skill stacks and trigger one-click resume screening directly for open positions.
- Screen Resumes Studio: Drag-and-drop runtime PDF resume ingestion with Apache PDFBox 3.0.8 text extraction and real-time Java DSA screening.
- Candidates Directory & CSV Export: Live search, quick skill filter pills ([Java], [Spring Boot], [SQL], etc.), score sorting, card/table views, and One-Click CSV Shortlist Export.
- Hiring Pipeline Kanban: Visual candidate tracking across stages: Shortlisted, Interview Scheduled, Under Review, and Rejected.
- Multi-Resume Comparator: Simultaneously compares 2, 3, 4, 5+ resumes side-by-side across match score, verified skills, and missing requirements.
- Duplicate Detection Auditor: Flags duplicate and near-duplicate submissions using SHA-256 digests and similarity percentages.

- Candidate Profile Dossier: Detailed modal dossier featuring match dials, verified/missing skill columns, pipeline stage selector, and recruiter notes.

6. REST API Specification

Method	Endpoint	Description	Status
GET	/api/health	Backend engine health check	200 OK
GET	/api/jobs	Retrieve active job requisitions	200 OK
POST	/api/jobs	Create new job opening	201 Created
GET	/api/resumes	Retrieve loaded candidate resumes	200 OK
POST	/api/resumes/upload	Upload runtime PDF resumes	200 OK
POST	/api/analyze	Trigger Java DSA screening pipeline	200 OK
GET	/api/candidates	Retrieve ranked candidates list	200 OK
GET	/api/candidates/{id}	Retrieve individual candidate profile	200 OK
POST	/api/compare	Multi-resume side-by-side comparison	200 OK
GET	/api/duplicates	Exact and near-duplicate audit report	200 OK
GET	/api/analytics	Recruiter dashboard summary metrics	200 OK

7. Setup & Execution Guide

The project can be executed seamlessly from Eclipse and terminal environments:

```
# 1. Running the Java DSA Backend (Terminal)
javac -d bin -cp "lib/*:bin" $(find src -name "*.java")
java -cp "lib/*:bin" com.resumex.api.ResumeXApiServer

# 2. Running the React Frontend (Terminal)
npm run dev      # Starts Vite server at http://localhost:5173

# 3. Running from Eclipse IDE
File -> Open Projects from File System -> Select ResumeX -> Run ResumeXApiServer.java
```